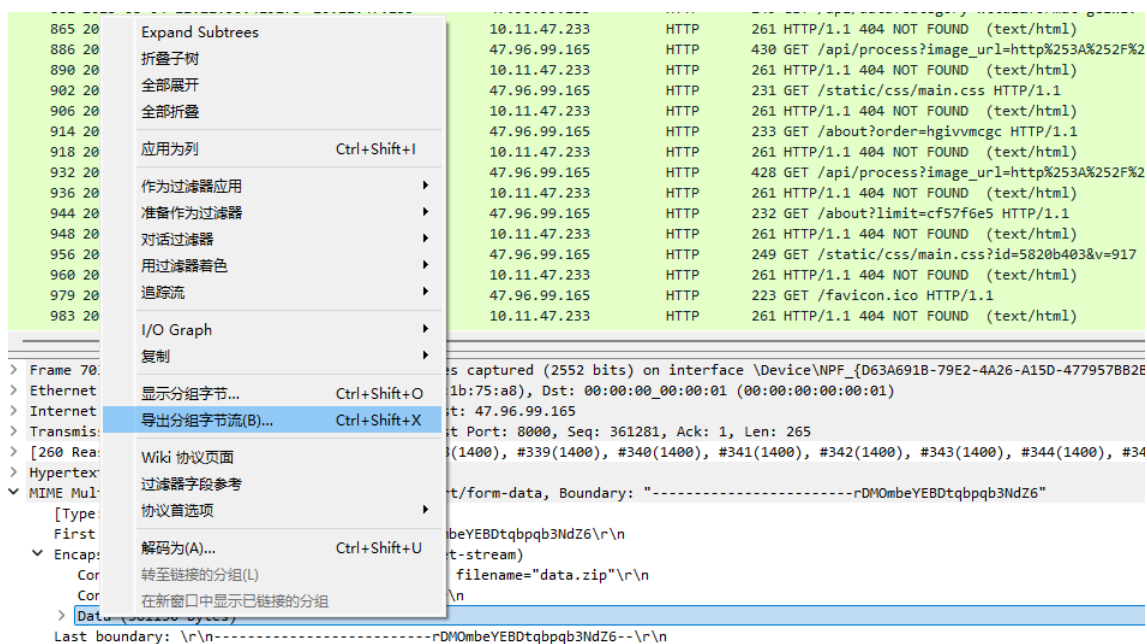



```

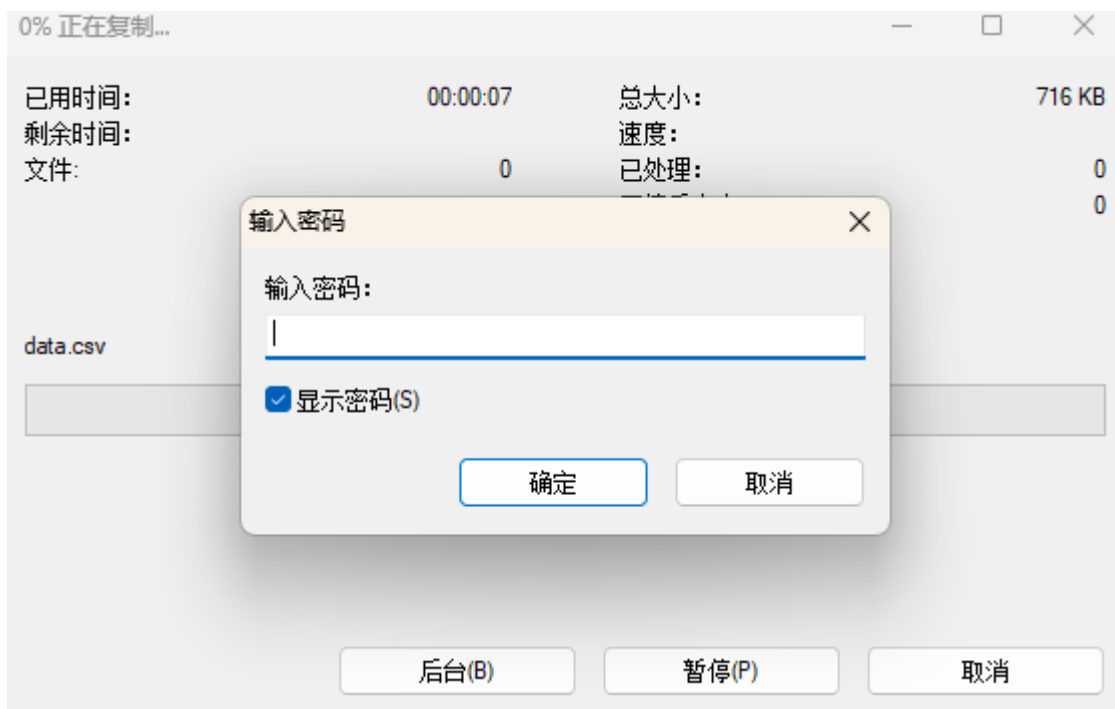
> Frame 703: 319 bytes on wire (2552 bits), 319 bytes captured (2552 bits) on interface \Device\NPF_{D63A691B-79E2-4A26-A15D-4779578B2E
> Ethernet II, Src: AzureWaveTec_1b:75:a8 (cc:47:40:1b:75:a8), Dst: 00:00:00_00:00:01 (00:00:00:00:00:01)
> Internet Protocol Version 4, Src: 10.11.47.233, Dst: 47.96.99.165
> Transmission Control Protocol, Src Port: 53890, Dst Port: 8000, Seq: 361281, Ack: 1, Len: 265
> [260 Reassembled TCP Segments (361545 bytes): #338(1400), #339(1400), #340(1400), #341(1400), #342(1400), #343(1400), #344(1400), #34
> Hypertext Transfer Protocol
  MIME Multipart Media Encapsulation, Type: multipart/form-data, Boundary: "-----rDMOmbeYEBDtqbpb3NdZ6"
    [Type: multipart/form-data]
    First boundary: -----rDMOmbeYEBDtqbpb3NdZ6\r\n
    Encapsulated multipart part: (application/octet-stream)
      Content-Disposition: form-data; name="file"; filename="data.zip"\r\n
      Content-Type: application/octet-stream\r\n\r\n
      Data (361130 bytes)
    Last boundary: \r\n-----rDMOmbeYEBDtqbpb3NdZ6--\r\n

```

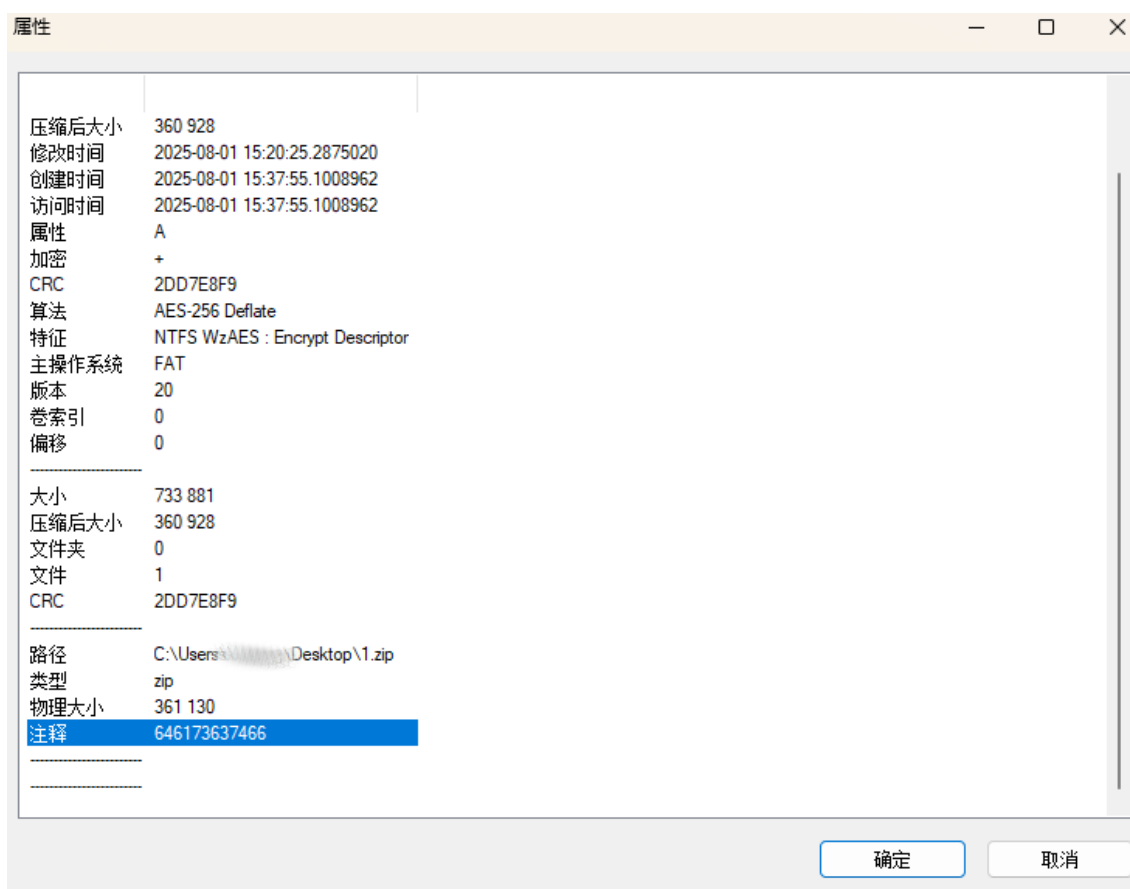
选中Data,右键导出分组字节流,注意要以压缩包的形式导出比如xxx.zip,操作如下图所示。



导出来之后尝试解压发现需要密码



随后，查看这个压缩包详细信息，发现注释有一串数字，转为字符串后是**dasctf**



```
http contains "dasctf"
```

```
tshark -r 1.pcap -Y 'http.request.method == "GET" && http.request.uri contains "dasctf"' -T fields -e http.request.full_uri > dasctf.txt
```

紧接着，利用以下poc.py提取和解码 dasctf 参数的值

```

chunks = []

with open(file_path, "r", encoding=encoding) as f:
    lines = f.readlines()

for line in lines:
    line = line.strip()
    if not line:
        continue

    parsed_url = urlparse(line)
    query = parse_qs(parsed_url.query)

    # 查找 image_url 参数并双重解码
    if "image_url" in query:
        encoded_inner_url = query["image_url"][0]
        inner_url = double_url_decode(encoded_inner_url)

        # 在内嵌 URL 中查找 dasctf 参数
        inner_parsed = urlparse(inner_url)
        inner_qs = parse_qs(inner_parsed.query)

        if param_name in inner_qs:
            chunks.extend(inner_qs[param_name])

return chunks

def main():
    file_path = "dasctf.txt"
    param_name = "dasctf"

    chunks = extract_dasctf_chunks(file_path, param_name)
    print(f"共提取到 {len(chunks)} 个 {param_name} 参数片段:\n")

    for i, chunk in enumerate(chunks, 1):
        print(f"Chunk {i}: {chunk}")

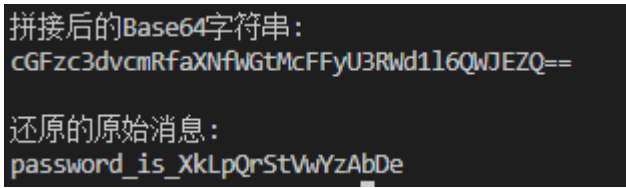
    full_base64 = ''.join(chunks)
    print(f"\n拼接后的Base64字符串:\n{full_base64}")

    try:
        original_message = base64.b64decode(full_base64).decode()
        print(f"\n还原的原始消息:\n{original_message}")
    except Exception as e:
        print(f"\nBase64解码失败: {e}")

if __name__ == "__main__":
    main()

```

运行结果如下图所示，拿到了压缩包密码**XkLpQrStVwYzAbDe**



对压缩包解密过后成功拿到data.csv。

#	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
1	姓名	身份证	手机号	银行卡号	IP地址																					
2	方浩	261914191.1.33E+10	12345678.46.211.233.111																							
3	李翔瑞	734817191.1.38E+10	022844946.247.217.208.107																							
4	毛捷浩红		1.38E+10	022821007.205.26.211.1																						
5	崔国健	594562191.1.78E+10	02230586.89.252.126.31																							
6	曹翔建峰	583214191.1.57E+10	02222527.192.168.1.256																							
7	傅黎	344773191.1.35E+10	022262137.192.168.1.256																							
8	周凯	543990191.1.77E+10	02220824118.120.14.252																							
9	彭鹏	941389191.1.78E+10	0228227912.66.202.220																							
10	黄雪	889438191.1.31E+10	02226243123.177.118.212																							
11	戴宇千	963000191.1.98E+10	02282553.59.200.164.237																							
12	吕子芳雯	392967200.1.83E+10	021333516218.196.53.184																							
13	李雨	778238191.1.36E+10	02267939.167.156.58.13																							
14	戴味轩芳	12345678.1.38E+10	02206809.96.33.48.32																							
15	马浩陈博	452170191.1.37E+10	123456785189.119.63.46																							
16	许伟英红	211137191.1.46E+10	02284487170.243.82.179																							
17	钱松凌洪	139033191.1.84E+10	02270353179.14.142.251																							
18	孙轩	383818191.1.86E+10	022305991183.78.104.166																							
19	许涛	341716191.1.78E+10	022821405106.121.67.228																							
20	曹辉子斌	497096191.1.54E+10	022833821.49.250.3.96																							
21	夏霞	107393200.1.38E+10	02290840.44.86.126.132																							
22	黄华浩洪	12345678.1.84E+10	0212268272.96.38.36																							
23	曹雨千王	398011191.1.47E+10	02287956.211.234.168.142																							
24	钟宏子王	669079191.1.97E+10	02276228133.27.173.224																							
25	陆瑶	123456191.1.73E+10	02282127511.60.202.48																							
26	陈凯	775216191.1.58E+10	0228448420.158.86.172																							
27	傅凯	623671191.1.50E+10	02230590.69.106.179.146																							
28	曹涛整华	597097191.1.86E+10	02282216167.27.219.49																							
29	曹宇千	522867191.1.84E+10	022703591216.157.139.55																							
30	高爽天轩	542252191.1.98E+10	02276279192.168.1																							
31	何佑	711643191.1.33E+10	0228220473.82.104.92																							
32	万万健	639897191.1.72E+10	02282330180.202.159.53																							
33	钟浩	297239191.1.71E+10	02282565.220.34.254.125																							
34	许艺彩	226740191.1.59E+10	02270051190.1.197.255																							
35	田国峰	667169191.1.69E+10	02222584140.95.94.9																							
36	毛霞凯	181450191.1.45E+10	02220236.289.117.221.151																							
37	汪志	542049191.1.76E+10	02220003.48.145.215.58																							
38	梁斌	964531191.1.54E+10	0227288578.127.127.70																							
39	郭然佳	924443191.1.33E+10	02123635.69.147.190.54																							
40	白熊思魏	171098191.1.74E+10	0222800528.21.204.237																							
41	莫雨	789502191.1.72E+10	0223056193.252.83.2																							
42	沈阳	869494191.1.34E+10	02276045140.85.209.103																							
43	王泽豪	750639191.1.38E+10	022761026141.160.224.244																							

然后根据个人信息规范文档，编写poc2.py清洗数据,拿到了清洗后数据answer.csv和不合规的数据1.csv

```
import csv
import re
import datetime

# 验证姓名 - 由于生成器确保所有姓名都是有效的，因此这个函数总是返回True
def validate_name(name):
    """
    验证姓名 - 由于生成器确保所有姓名都是有效的，此函数总是返回True
    """
    return True

# 验证身份证号码
def validate_id_card(id_card):
    """
    验证身份证号码是否符合规范：
    1. 18位二代居民身份证
    2. 前6位为数字（模拟区域编码，不验证实际有效性）
    3. 出生日期有效
    4. 校验码正确
    """
    if not id_card:
        return False

    # 检查长度和基本格式
```

```

if not re.match(r'^\d{17}[\dX]$', id_card):
    return False

# 检查出生日期是否有效
try:
    year = int(id_card[6:10])
    month = int(id_card[10:12])
    day = int(id_card[12:14])

    # 检查日期是否在合理范围内
    if not (1960 <= year <= datetime.datetime.now().year):
        return False
    if not (1 <= month <= 12):
        return False

    # 检查每月天数
    max_days = [0, 31, 29 if (year % 4 == 0 and year % 100 != 0) or year % 400
    == 0 else 28,
                31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
    if not (1 <= day <= max_days[month]):
        return False
except:
    return False

# 验证校验码
try:
    # 加权因子
    weights = [7, 9, 10, 5, 8, 4, 2, 1, 6, 3, 7, 9, 10, 5, 8, 4, 2]
    # 校验码映射
    check_codes = '10X98765432'

    # 计算加权和
    weighted_sum = sum(int(id_card[i]) * weights[i] for i in range(17))
    # 计算校验码
    check_code = check_codes[weighted_sum % 11]

    # 检查校验码是否匹配
    return id_card[17].upper() == check_code
except:
    return False

# 验证手机号码
def validate_phone(phone):
    """
    验证手机号码是否符合规范：
    1. 11位数字
    2. 真实的中国手机号码格式
    """
    if not phone:
        return False

    # 检查是否为11位数字

```

```

if not re.match(r'^\d{11}$', phone):
    return False

# 所有手机号都被视为有效，因为生成器只生成有效号段
return True

# 验证银行卡号
def validate_bank_card(card_number):
    """
    验证银行卡号是否符合规范：
    1. 13-19位数字
    2. 满足Luhn算法校验
    """
    if not card_number:
        return False

    # 检查是否只包含数字
    if not re.match(r'^\d+$', card_number):
        return False

    # 检查长度是否在13-19位之间
    if not (13 <= len(card_number) <= 19):
        return False

    # 使用Luhn算法验证
    def is_luhn_valid(card_number):
        sum_digits = 0
        card_number_reversed = card_number[::-1] # 反转卡号

        for i, digit in enumerate(card_number_reversed):
            digit = int(digit)
            # 偶数位（从1开始）需要翻倍
            if i % 2 == 1:
                digit *= 2
                if digit > 9:
                    digit -= 9
            sum_digits += digit

        # 卡号是有效的，如果总和是10的倍数
        return sum_digits % 10 == 0

    return is_luhn_valid(card_number)

# 验证IP地址
def validate_ip(ip):
    """
    验证IP地址是否符合规范：
    1. 标准IPv4地址格式
    2. 每段数字在0-255之间
    3. 四段，以点号分隔
    4. 不允许前导零
    """

```



```

    if not ip:
        return False

    # 分割IP地址
    parts = ip.split('.')

    # 检查是否为4段
    if len(parts) != 4:
        return False

    # 检查每段是否为有效数字
    for part in parts:
        # 检查是否为数字
        if not part.isdigit():
            return False

        # 检查是否有前导零
        if len(part) > 1 and part[0] == '0':
            return False

        # 检查数值范围
        value = int(part)
        if not (0 <= value <= 255):
            return False

    return True

# 验证记录是否合规
def validate_record(record):
    """
    验证一条记录是否符合所有规范
    """
    is_valid = (
        validate_name(record['姓名']) and
        validate_id_card(record['身份证']) and
        validate_phone(record['手机号']) and
        validate_bank_card(record['银行卡号']) and
        validate_ip(record['IP地址'])
    )
    return is_valid

# 主函数
def main(input_file='data.csv', valid_output='answer.csv', invalid_output='1.csv'):
    # 读取CSV文件
    valid_records = []
    invalid_records = []

    try:
        with open(input_file, 'r', encoding='utf-8') as f:
            reader = csv.DictReader(f)
            fieldnames = reader.fieldnames

```

```

# 处理每条记录
for record in reader:
    if validate_record(record):
        valid_records.append(record)
    else:
        invalid_records.append(record)

# 写入有效记录
with open(valid_output, 'w', newline='', encoding='utf-8') as f:
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(valid_records)

# 写入无效记录
with open(invalid_output, 'w', newline='', encoding='utf-8') as f:
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(invalid_records)

print(f"数据清洗完成!")
print(f"有效记录数量: {len(valid_records)}")
print(f"无效记录数量: {len(invalid_records)}")
print(f"有效记录已保存至: {valid_output}")
print(f"无效记录已保存至: {invalid_output}")

except Exception as e:
    print(f"处理过程中发生错误: {e}")

if __name__ == "__main__":
    main()

```

最后上传answer.csv，当校验达标（98%）以上时，将给出此题 flag。

验证码区: (请输入验证码, 验证码不对则文件无法成功上传)

AA89  [点击刷新验证码](#)

文件上传区: (请在此处上传文件)



Drop file here or [click to upload](#)

只允许上传 csv 文件格式 (其中文件名称随意)

点击下方下载示例文件

-  answer.csv
-  answer.csv
-  answer.csv
-  answer.csv

结果展示区: (请注意, 刷新页面结果将清空)

文件名	上传状态	上传相关备注	分数(百分比)	FLAG
answer.csv	1	上传成功	0.005%	give_you_flag_when_score>98%
answer.csv	1	上传成功	100.000%	DASCTF{7b43c28f19d5e6a2b0f8c4d7e3a9f156}
answer.csv	1	上传成功	100.000%	DASCTF{f1e8a7b2c93d405e9a6bcd21e74f3a60}